Modular Development

Modular Development, Building, Make, and Testing

Liting Zhang

University of West Florida

Overview

- Modular development: drivers and reusable modules
- C++ build pipeline: preprocessing, compiling, linking
- Building with g++: one-step vs step-wise compilation
- Make: targets, dependencies, recipes, and automation
- Testing: test drivers, assert, and unit testing frameworks

Learning goals

After this lecture, you should be able to

- Organize a project into drivers and modules using header and source files
- Explain what preprocessing, compiling, and linking do
- Build modular projects using step-wise compilation and linking
- Write a basic makefile with multiple targets
- Write and run simple unit tests using assert or a framework style

Core idea: build and test faster as projects grow

- Separate code into modules so changes stay local
- Compile modules separately so rebuilds are fast
- Use make so the build is repeatable and correct
- Use tests so behavior stays correct after changes

Modular Development

What is modular development

Modular development in C++ means splitting a program into independent, reusable components.

- A module has a clear responsibility
- A header contains declarations (the interface)
- A source file contains definitions (the implementation)
- A driver contains main and builds an executable

Benefits

- Reuse the same module in many programs
- Multiple people can work on different modules at the same time
- Smaller rebuilds: recompile only what changed
- Easier testing: test a module with a separate test driver
- Cleaner code: fewer giant files, clearer responsibilities

Drivers vs modules

Driver

- Has main
- Produces an executable
- Often one driver per program or per test executable

Module

- No main
- Provides reusable classes and functions
- Usually a pair: something.hpp and something.cpp

Header guards

```
1  #ifndef THREE_INTS_HPP
2  #define THREE_INTS_HPP
3
4  class ThreeInts {
5  public:
6    ThreeInts(int a, int b, int c);
7    int sum() const;
8  private:
9    int x_, y_, z_;
10 };
11
12 #endif
```

Practice 1: classify files

Project files:

- main.cpp
- table.hpp, table.cpp
- list.hpp, list.cpp

Task:

- Which file is the driver
- Which files are modules

Practice 1 Answer

- Driver: main.cpp (contains main)
- Modules: table.hpp with table.cpp, list.hpp with list.cpp

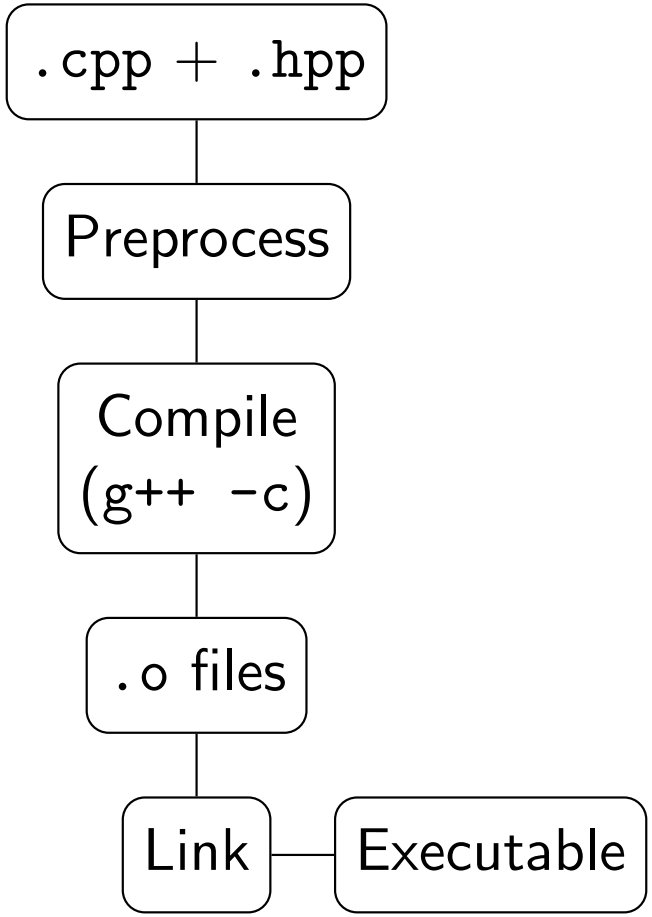
Build Pipeline and g++

C++ build pipeline

Building translates source code into an executable.

- Preprocessing: expands includes and macros
- Compiling: turns each source file into an object file
- Linking: combines object files into an executable

Pipeline picture



Headers are included into source files; headers are not compiled by themselves.

g++ options you will use

- -std=c++14 (or newer)
- -Wall -Wextra
- -I include
- -c
- -o name

One-step build

```
1 g++ -std=c++14 -Wall -Wextra -I include \
2   -o main main.cpp table.cpp list.cpp
```

This compiles and links in one command.

Step-wise build

Compile each source file into an object file:

```
g++ -std=c++14 -Wall -Wextra -I include -c main.cpp
g++ -std=c++14 -Wall -Wextra -I include -c table.cpp
g++ -std=c++14 -Wall -Wextra -I include -c list.cpp
```

Link object files:

```
g++ -o main main.o table.o list.o
```

Practice 2: what is wrong

```
1 g++ -o main main.cpp grade-book.cpp grades.hpp
```

- What is wrong (if anything)
- Is this one-step or step-wise

Practice 2 Answer

- This is one-step (compile and link together).
- The header file should not be listed as a build input.
- Better: g++ -o main main.cpp grade-book.cpp

Practice 3: object file mistake

```
1 g++ -o main.o main.cpp
```

- What does this likely produce
- How do you correctly create main.o

Practice 3 Answer

Without -c, this links and creates an executable named main.o.
Correct:

```
1 g++ -std=c++14 -Wall -Wextra -c main.cpp -o main.o
```

Practice 4: -c with -o mistake

```
1 g++ -c -o main.o main.cpp grade-book.cpp
```

- Why is this incorrect
- Show one correct fix

Practice 4 Answer

-c compiles each source file into its own object file. Using -o main.o tries to name a single output for multiple inputs.
Fix by compiling separately:

```
1 g++ -c main.cpp -o main.o
2 g++ -c grade-book.cpp -o grade-book.o
3 g++ -o main main.o grade-book.o
```

Make

Why use make

Step-wise builds have repeated commands. make automates:

- build order
- rebuild only what changed
- shared compiler flags
- multiple executables (main and tests)

Make vocabulary

- Target: output to build
- Dependencies: inputs needed for the target
- Recipe: commands to build the target

Automatic variables:

- $@: target name
- $^ all dependencies
- $< first dependency

Makefile example: two executables

Example: m02-multimodule/makefile

```
1  CXX      := g++
2  CXXFLAGS := -std=c++14 -Wall -Wextra -I include
3  all: main test
4
5  main: main.o three-ints.o
6   $(CXX) -o $@ $^
7  test: test.o three-ints.o
8   $(CXX) -o $@ $^
9  main.o: main.cpp include/three-ints.hpp
10  $(CXX) $(CXXFLAGS) -c $< -o $@
11 test.o: test.cpp include/three-ints.hpp
12  $(CXX) $(CXXFLAGS) -c $< -o $@
13 three-ints.o: three-ints.cpp include/three-ints.hpp
14  $(CXX) $(CXXFLAGS) -c $< -o $@
15
16 .PHONY: clean
17 # test compiled exectuable, *.dSYM debug symbol folders
18 clean :
19  rm -rf test main *.o *.gc* test *.dSYM
```

Common make commands

- make
- make main
- make test
- make clean

Testing

Why testing

- Manual testing does not scale
- Tests describe expected behavior
- Tests catch regressions after changes
- Modules are easier to test than monoliths

Types of tests

- Unit tests: one function or class
- Integration tests: modules working together
- System tests: full program behavior

Simple test driver

```
1 #include "three-ints.hpp"
2 #include <iostream>
3
4 int main() {
5   ThreeInts t(1, 2, 3);
6   std::cout << "sum=" << t.sum() << "\n"; // expected 6
7 }
```

Tests with assert

```
1  #include "store-item.hpp"
2  #include <cassert>
3
4  void testStoreItem() {
5    StoreItem a("Apple");
6    StoreItem b("apple");
7    assert(a.equals(b));
8  }
9
10 int main() {
11   testStoreItem();
12 }
```

Framework style (Catch2-like)

```
1  #include "catch.hpp"
2  #include "store-item.hpp"
3
4  TEST_CASE("store item equality", "[storeItem]") {
5    SECTION("case-insensitive comparison") {
6      StoreItem a("Apple");
7      StoreItem b("apple");
8      CHECK(a.equals(b));
9    }
10 }
```

Run tests from make

```
1 test-run: test
2   ./test
```

```
1 make test-run
```

Review

Summary

- Modules separate interface (hpp) from implementation (cpp)
- Step-wise builds compile to object files, then link
- make automates dependencies and rebuilds only what changed
- Tests reduce regressions and increase confidence in correctness

Review questions

- Why are header guards necessary
- Why is step-wise compilation faster for iterative changes
- What are target, dependencies, and recipe in make
- What is the difference between a test driver and a unit test framework